

Food-Delivery Aggregator – System Design Project Report

Submitted By:

Name: Eshan Thakur

Roll No.: 23

Registration No.: 12410061

Course: B .Tech in Generative AI

Semester: 3

Session: 2024–2025

College: Lovely Professional University

1. Introduction

Food-ordering platforms like Zomato, Swiggy, Uber Eats, and DoorDash have shaped the way in which individuals order food. While customers expect speedier delivery, live tracking, and secure modes of payment, restaurants and delivery partners use the platform for stable and equitable distribution of orders.

The project mainly focuses on designing a Food-Delivery Aggregator System that will enable customers to browse menus, place orders, get real-time updates, and track the delivery partner live on a map.

2. Requirements Pack

2.1 Stakeholder Analysis

Stakeholder What they need

Customers Smooth ordering, fast delivery, real-time tracking

Restaurants: Clear order details and timing for food preparation

Delivery Partners Optimized routes, fair assignments, timely payout

Operations Team: Monitor delays, resolve issues, manage surge load

Payments/Finance: Secure transactions, settlements, refunds.

Customer Support View into orders for dispute resolution

2.2 MoSCoW prioritization

Priority	Requirement
MUST HAVE	Order placing, driver assignment, live tracking, payments
SHOULD HAVE	Surge pricing, cached menus
COULD HAVE	Promo codes, loyalty points
WON'T HAVE ?	Drone/robot delivery: out of scope

3. Functional Requirements

Customer

- Registration/Login
- Browse/search restaurants and menus
- Items can be added to the cart and ordered.
- Track delivery partner live

Get notifications: push/SMS/email

Restaurant

- Manage menu items and prices
- Accept/reject orders
- Update food preparation status

Delivery Partner (Driver)

- Delivery request
- Proceed to restaurant and customer
- Share GPS location continuously

System Functions

- Intelligent driver assignment algorithm
- ETA calculation and updates
- Secure payment processing

4. Non-Functional Requirements

NFR Type	Target
----------	--------

Availability 24/7 uptime, 99.95% reliability

Performance\tLive ETA & location latency < 1 sec

Scalability: 10,000+ concurrent deliveries

Security OAuth2 + JWT, TLS Encryption

Consistency: Strong for payments, eventual for tracking

Maintainability Microservices, domain-based boundaries

5. Constraints & Assumptions

- Uses third-party map APIs - Google Maps
 - Drivers face network instability
 - Eventual consistency acceptable for ETA & location
 - Payments require strong consistency
 - Menu items can be cached and might not be updated immediately everywhere
-

6. User Stories (Human-Centered)

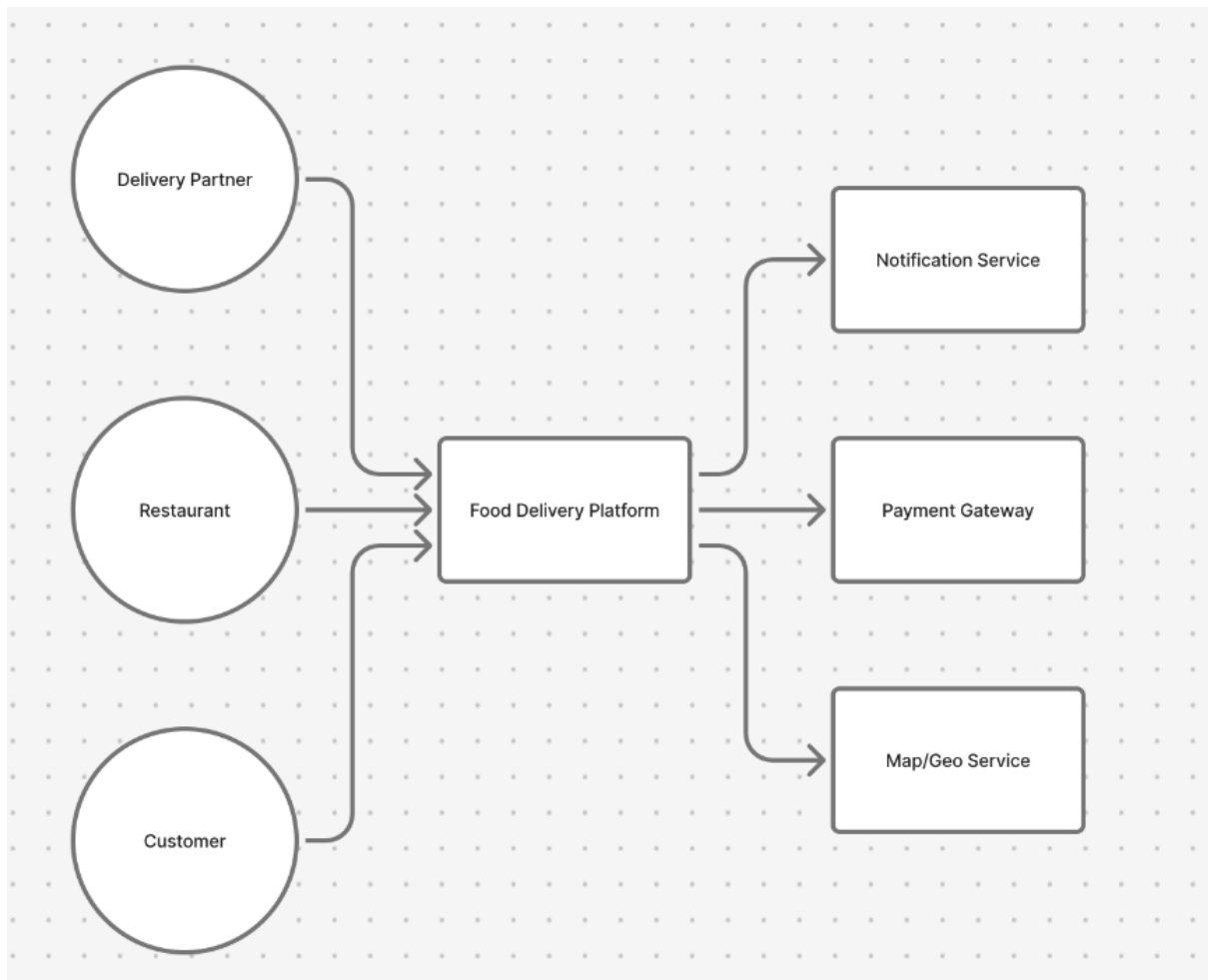
Customer: I want to track my delivery in real time so I know when my food will arrive.

Restaurant: I want orders to reach me instantly so that no preparation of food gets delayed.

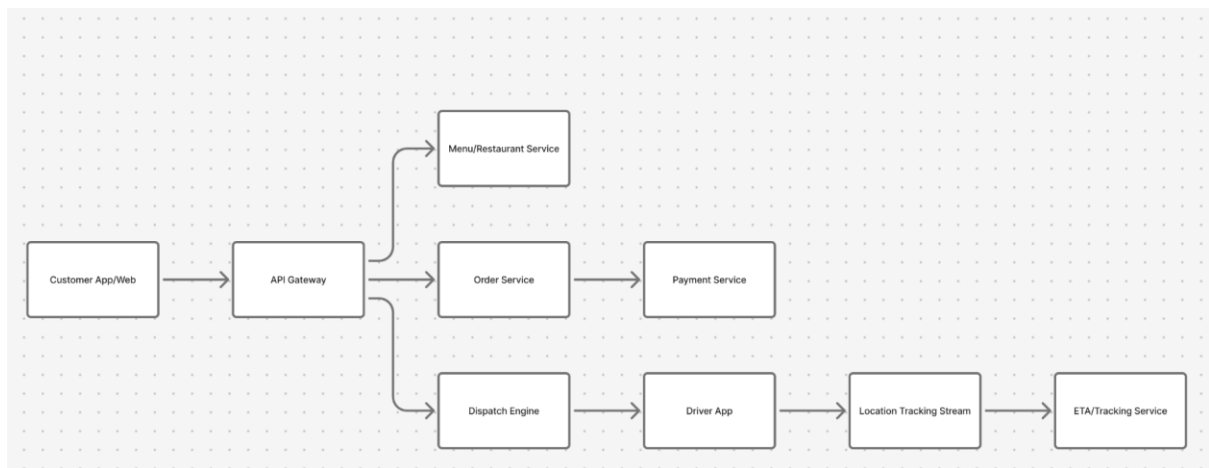
✈ Driver: I want the best route so that I can deliver efficiently and earn more.

Operations Manager: I want to see active orders so that I can resolve delivery issues quickly.

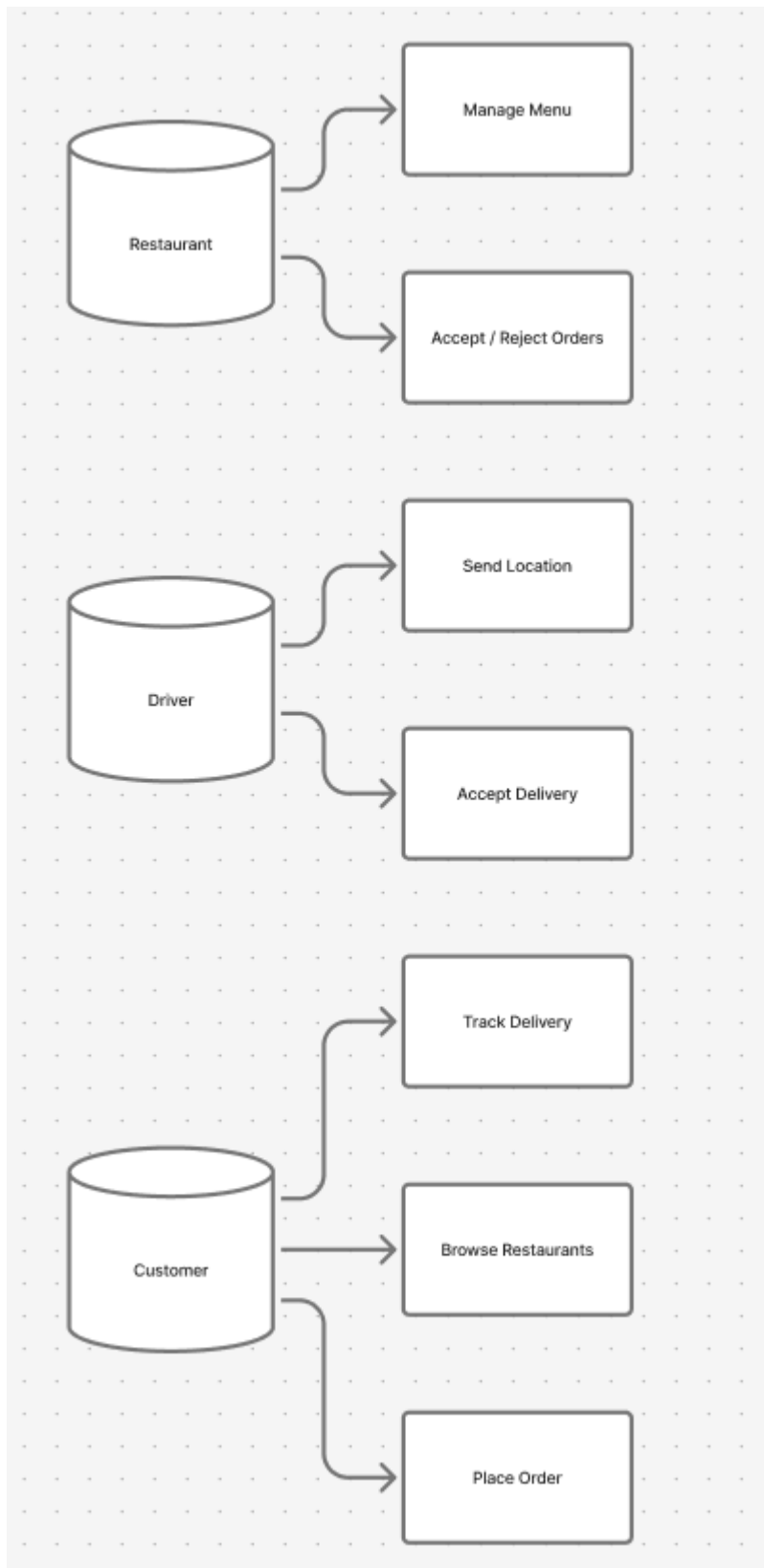
7. System Design Diagrams



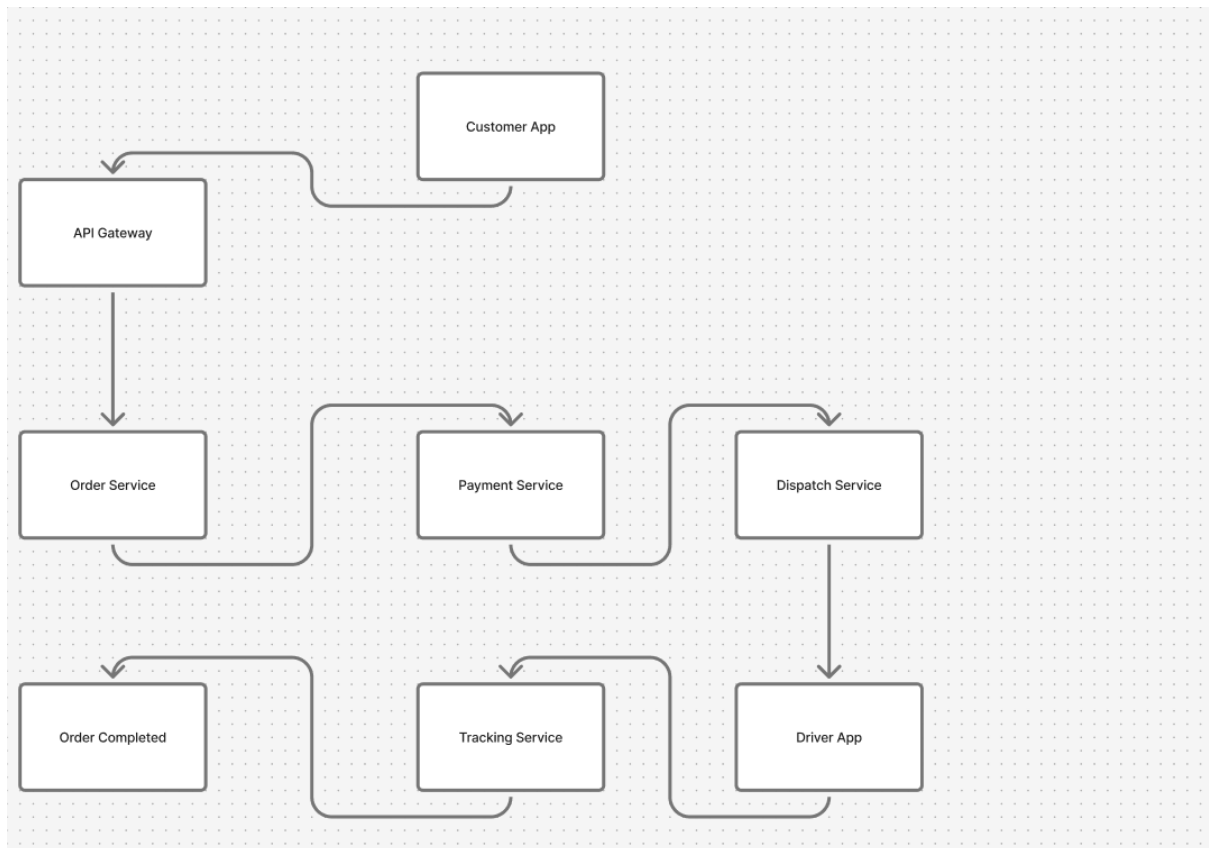
7.1 System Context Diagram



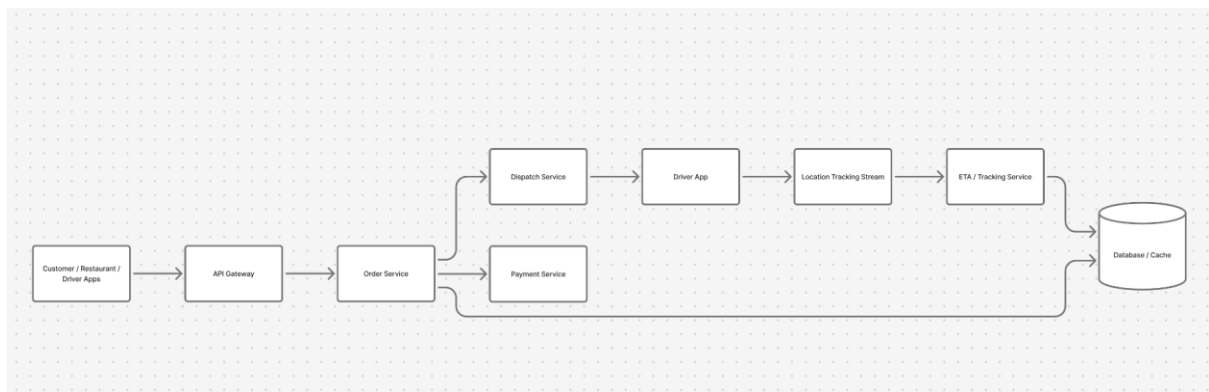
7.2 Use Case Diagram



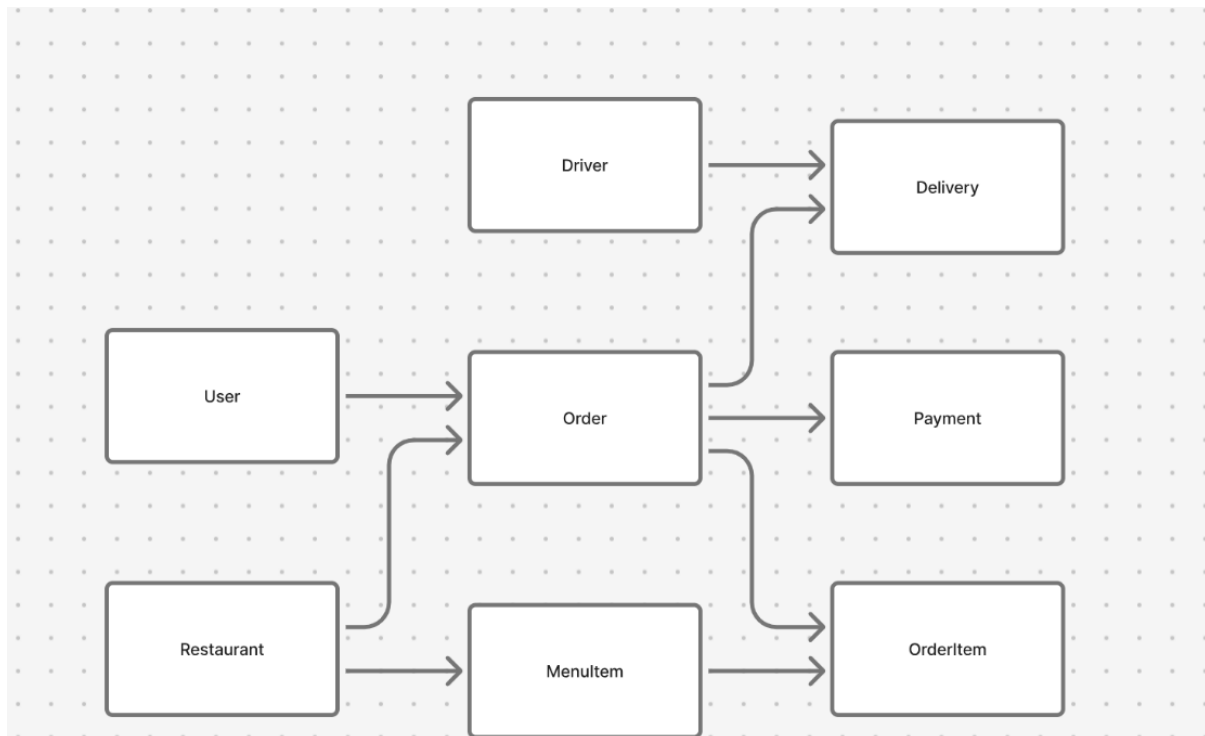
7.3 Sequence Diagram (Order → Dispatch → Delivery Flow)



7.4 High-Level Architecture



7.5 ER / Data Model Diagram



8. Engineering Notes

8.1 Capacity Planning

On average, each driver sends one location update every second.

With 10,000 active deliveries, this means:

10,000 GPS events per second

~600,000 per minute ????

~864 million per day

Therefore, the system requires a high-volume streaming platform such as Kafka or Pulsar to manage location events.

8.2 API Specifications (Sample)

Method	Endpoint	Description
--------	----------	-------------

POST	/order	Create a new order
------	--------	--------------------

POST	/dispatch/assign	Assign best driver
------	------------------	--------------------

GET	/track/{order_id}	Real time tracking
-----	-------------------	--------------------

GET	/restaurants	Search nearby restaurants
-----	--------------	---------------------------

8.3 Data Model (Tables Overview)

Suggested entities:

- **USER**
- **Restaurant**
- **MenuItem**
- **Order**
- **Order Item**
- **Courier/Driver**
- **Delivery**
- **Location Update**

8.4 Consistency Choices

Component Model Reason

Payments Strong cannot risk double-charging

Orders Strong Order state must remain valid

ETAs/Tracking Eventual Minor delay acceptable

Menu Eventual Cached for better user experience

8.5 Caching & Indexing

- **Redis cache for menu & ETA snapshots**

CDN for restaurant images

- **Geo-index for nearby driver matching**

8.6 Rate Limiting & Resiliency

- **Max 2 orders/min per user**
- **Exponential backoff retry**
- **Circuit breakers for map & payment service**

- Dead-letter queues for failed events
 - Multi-region failover support
-

8.7 Observability

- Logs → errors, delays, failed payments
 - Metrics → latency, API load, driver backlog
 - Traces → full order journey (OpenTelemetry)
-

8.8 Maintenance Plan

Daily DB Backups

- Canary rollout CI/CD
 - Log retention: 7–14 days
 - Automated scaling during rush hours
-

9. Quality Objectives (SLOs)

Metric Target

Uptime 99.95%

Order placement latency < 300ms

Tracking update latency < 1 second

Driver dispatch time < 3 seconds

10. Trade-Off Considerations

Decision Benefit Cost

WebSockets for tracking True real-time Higher connection overhead

Cached menus Faster app experience. Menu updates delayed

Microservices Independent scaling More DevOps complexity

Eventual ETA Fast + scalable Small accuracy delay

11. Conclusion

This system design will help to show how a real-world food-delivery aggregator can be built to operate efficiently at scale while handling real-time delivery tracking, peak-hour load, and secure financial transactions. Using microservices, streaming systems, caching, and geo-sharding will allow the platform to serve metropolitan cities with thousands of concurrent deliveries, much like Zomato, Swiggy, Uber Eats, and DoorDash.